المحاضرة — شرح الكود بالتفصيل

مرجع فني: كل ملف بيعمل إيه، مين بينادي مين، النوع اللي بيرجع، والكاستينج. مكتوب للكود بعد الـ P0 pass.

0) الصورة الكبيرة

Clean Architecture + Feature-first. الاتجاه: `Presentation → Domain → Data`. الـ Presentation ميعرفش Supabase، والـ Domain ميعرفش مين بينفذ، والـ Data هو الوحيد اللي بيتكلم مع السيرفر.

قاعدة 1: Either<Failure, T>

```
res.fold(
  (failure) => ... , // Left
  (data)    => ... , // Right
);
```

معناها: دالة واحدة بتتنفذ، حسب الحالة. كل repository بيرجع Either، ومفيش exception بيطلع برّه الـ data layer.

قاعدة 2: States sealed + Equatable

`sealed` بتخلي الـ switch إجباري يغطي كل الحالات. `Equatable` بتخلي البلوك ميبعتش state مساوية للقديمة.

1) main.dart — من أول نفس

1. **WidgetsFlutterBinding.ensureInitialized**(): تشغيل الـ engine قبل أي await.
2. **ScreenUtil.ensureScreenSize**(): قراءة أبعاد الشاشة قبل أول frame، عشان `.w/.h/.sp` تشتغل.
3. **Supabase.initialize(url, publishableKey)**: اتصال + إنشاء `Supabase.instance.client` + قراءة الـ session المحفوظة + تشغيل `onAuthStateChange`.
4. **Bloc.observer**: لوجات debug فقط، أنواع الأحداث مش محتواها (الباسورد مش بيتطبع).
5. **EasyLocalization.ensureInitialized**(): تحميل الترجمات واللغة المحفوظة (المفتاح `locale`).
6. **configureDependencies()**`()`: `getIt.init` — تسجيل كل الكلاسات من `injection.config.dart`.
7. **runApp**: EasyLocalization → MultiBlocProvider (Locale, Theme, Auth, Character) → ElmohadraApp.

ElmohadraApp.initState

```
_authNotifier = AuthNotifier(getIt<AuthBloc>());
_router = appRouter(_authNotifier);
context.read<AuthBloc>().add(CheckAuthStatusEvent());
AppLinks().uriLinkStream.listen(... DeepLinkReceivedEvent ...);
```

الـ router والـ notifier بيتعملوا مرة واحدة، و`dispose` بيقفلهم ويلغي الـ subscription.

build

```
ScreenUtilInit(designSize: Size(390, 900)) → MaterialApp.router(
  theme: AppTheme.light, darkTheme: AppTheme.dark,
  themeMode: context.watch<ThemeCubit>().state.flutterThemeMode,
  routerConfig: _router,
  locale: context.locale,
)
```

`w.` = القيمة × عرض الشاشة ÷ 390، و`h.` بنسبة 900، و`sp.` للنص مع minTextAdapt.

2) الراوتينج

GoRouter

- `initialLocation: /splash`.
- `refreshListenable: authNotifier`: أي state جديد للـ AuthBloc → إعادة تقييم redirect.

  مفيش `context.go` بعد تسجيل الدخول.
- ShellRoute بتحيط 4 صفحات بـ MainShell (BottomNav + FAB ثابتين).
- signin/signup/splash برّه الـ shell، وكل واحد بيعمل البلوكات بتاعته.
- في `buildPageTransition`: CustomTransitionPage، fade + scale (1.15 → 0.98 → 1.0)

  280ms.

جدول GoRouterState

الخاصية	النوع	المعنى بدقة	مثال على /signup?email=a@b.com
`state.uri`	Uri	العنوان الكامل مع query	/signup?email=a@b.com
`state.matchedLocation`	String	المسار الذي تمت مطابقته، بدون query ولا fragment	/signup
`state.fullPath`	String ?	نمط الراوت كما هو معرّف (بالـ :params)	/signup
`state.pathParameters`	Map<String, String>	قيم :id من النمط	{}
`state.queryParameters`	Map<String, String>	قيم بعد علامة ؟	{email: a@b.com}
`state.extra`	Object ?	كائن عشوائي يُمرر مع go(extra:) ولا يبقى بعد إعادة التشغيل	—

`state.name`	String ?	اسم الراوت	signup

دلالات redirect بدقة

- التوقيع: `FutureOr<String?> Function(BuildContext, GoRouterState)`.
- بيتنفذ: عند البداية، عند كل go/push، وعند كل notify من refreshListenable.
- `null` = اسمح/خليك مكانك. `String` أو `Uri` = انتقل لهناك (وتعاد دورة redirect على الهدف).
- لو رجّعت نفس المكان: GoRouter بيعتبرها لا تغيير لمنع الحلقة.
- الـ state اللي بتوصلك هي **الهدف** مش الشاشة الحالية.
- الترتيب: redirect العام أولاً، ثم redirect الخاص بالراوت.

خطأ شائع: المقارنة بـ `state.uri.toString()` بدل `matchedLocation` — الـ query بيلخبط المقارنة وتدخل حلقة redirect

منطق التطبيق

الحالة	النتيجة
AuthInitial / AuthLoading	null (سيب المستخدم مكانه)
AuthAuthenticated	لو رايح signin/signup → /home
AuthNewUser	لو مش على signin/signup → /signup?email=..&id=..

AuthUnAuthenticated لو مش على /signin ← signin/signup

AuthError /signin

3) طبقة الـ Domain

- **UserEntity**: بيانات المستخدم + Equatable، `UserStatus {active, banned}`.
- **AuthUserResult**: existingUser / newUserId / newUseremail — حاملة قرار للـ AuthBloc.
- **SigninParams / SignupParams**: باراميترات (password nullable لمستخدمي جوجل).
- **BaseUsecase<R,P>** + `NoParams`: كل usecase بي`call` implement فيبقى قابل للنداء كدالة.
- **AuthRepository**: 7 دوال كلها `<<Future<Either<Failure, X`.

4) طبقة الـ Data والكاستينج

UserModel.fromJson

```
id: json['id'] as String,
academicYear: json['academic_year'] as int,
createdAt: DateTime.parse(json['created_at'] as String),
status: UserStatus.values.firstWhere((v) => v.name == json['status'],
        orElse: () => UserStatus.active),
```

القيم من Supabase هي `dynamic`؛ `as` كاست صارم بيرمي TypeError لو النوع مختلف. الوصول لـ `Map` واحد بـ

`maybeSingle()` أنضف من `res[0]`.

signInWithEmail

```
auth.signInWithPassword(...) → res.user
if (user == null) throw AuthException(code: 'invalid_credentials')
profile = _fetchProfile(user.id)  // users.maybeSingle
if (profile == null) throw AuthException(code: 'user_not_found')
```

signUpWithEmail

```
password != null ? auth.signUp(...) : currentUser.id   // جوجل
if (currentSession == null) throw AuthException(code: 'email_not_confirmed')
users.insert({...}).select().single() → UserModel
```

الأعمدة | /level/xp/coins... بتيجي من العميل. | RLS بتحمي الصفوف مش القيم. الحل: | defaults

trigger + | + منع الكتابة في أعمدة اللعب.

checkEmailIsExist

```
functions.invoke('check-email-exists', body: {'email': email})
return res.data['exists'] as bool;
```

AuthRepositoryImpl._guard

```
Future<Either<Failure, T>> _guard<T>(Future<T> Function() call) async {
  try { return Right(await call()); }
  catch (e) { return Left(FailureMapper.map(e)); }
}
```

Exception	Failure
AuthException	AuthFailure(code)
PostgrestException / StorageException	ServerFailure
SocketException	NetworkFailure
أي حاجة تانية	ServerFailure

5) AuthBloc

Event	اللي بيحصل
CheckAuthStatusEvent	Loading → getCurrentUser → Authenticated / NewUser / UnAuthenticated
DeepLinkReceivedEvent	Loading → (لو مفيش session: انتظار signedIn بـ timeout 10s) → getCurrentUser → نفس القرار مع isGoogleSignIn
HandleSigningWithGoogleEvent	فتح البراوزر، بدون Loading (لو المستخدم قفل البراوزر ميفضلش معلق)

SignOutEvent	Loading → signOut → UnAuthenticated

6) تسجيل الدخول

```
onTap → AppValidators → SignInEvent(SigninParams)
→ SigninWithEmailUsecase → AuthRepository._guard → datasource → Supabase
→ Right/Left → SigninBlocSuccess/SigninBlocError
→ toast + 500ms + CheckAuthStatusEvent
→ AuthAuthenticated → redirect /home
```

7) Signup Wizard

شاشة واحدة، int `currentScreen`، و`AnimatedSwitcher`. كل خطوة StatefulWidget مكشوف الـ State،

والأب بيمسكها بـ `GlobalKey<XState>` وينادي `isDisabled` / `validateInputs()`.

```
didChangeDependencies:
  uri = GoRouterState.of(context).uri
  isGoogle = email/id موجودين في queryParameters
  google → [Name, Uni, Character, Summary]
  عادي  → [Credential, Name, Uni, Character, Summary]
```

زرار التالي

```
isValidated → update للـ Cubit → currentScreen++
Credential استثناء: CheckEmailIsExist → listener يتقدم عند isExist == false
آخر خطوة: SignUpEvent(SignupParams) → SignupWithEmailUsecase → ... → SignUpSuccess
```

ليه didChangeDependencies مش initState

لأن `GoRouterState.of(context)` بيتسجل كاعتماد على InheritedWidget. في `initState` الاعتماد على

InheritedWidget ممنوع (assert). الحل: didChangeDependencies مع حارس `if (screens.isEmpty)` لأنها بتتنادى أكتر من مرة.

8) feature الشخصيات

```
initState → CharacterBloc.add(GetCharactersEvent)
→ CharacterLoading → GetCharactersUsecase(NoParams) → repo → datasource
→ from('characters').select() → List<Map>
→ .map(CharacterModel.fromJson).toList() → List<CharacterModel>
→ Right(List<CharacterEntity>)  // covariance
→ GetCharactersSuccess / CharacterIError
```

الشاشة: grid 3 أعمدة، الشخصية المقفولة عليها طبقة سوداء + قفل، والزرار بيتقفل. الصور عبر `AppNetworkImage` (placeholder + error).

9) طبقة core

الملف	المحتوى
app_colors	ألوان dark/light + accent/success/warning/error
app_typography	Lalezar/Cairo للعربي و Russo One/Exo 2 للإنجليزي، كلها sp.
app_spacing	4 / 8 / 16 / 24 / 32 / 48
app_radius	8 / 16 / 24 / 32 / 100
app_duration / app_curves /	توكنز الحركة والظل

app_shadows	
app_theme	ColorScheme صريح + cardTheme + appBarTheme
locale_cubit / theme_cubit	حفظ في SharedPreferences + emit
AppTextField	FocusNode + border متحرك + إظهار/إخفاء الباسورد + errorText خارجي
AppButton	3 أنواع + isLoading + isDisabled
AppToast	toastification.showCustom
AppAnimatedEntry	fade + slide مع delay
AppNetworkImage	CachedNetworkImage + placeholder + error

10) جدول الكاستينج

من	إلى	الطريقة
Map<String,dynamic>	UserModel	as / DateTime.parse / firstWhere
PostgrestList	List<CharacterModel>	map + toList
List<CharacterModel>	List<CharacterEntity>	covariance (وراثة)

Exception	Failure	FailureMapper.switch
Future<T>	Either	guard try/catch _
Either	State	fold
AuthStates	Route	sealed switch

11) أسئلة وأجوبة (مرجع سريع)

initState vs didChangeDependencies

	initState	didChangeDependencies
عدد المرات	مرة واحدة	مرة بعد initState + كل مرة تتغير فيها dependency
InheritedWidget	ممنوع	مسموح
الاستخدام	Controllers / FocusNode / أول event	Theme.of / MediaQuery / GoRouterState / context.locale

ليه لا نستخدم rethrow وخلاص

rethrow في catch فاضي = كود ميت. الـ datasource بيرمي exceptions الأصلية من Supabase، والـ repository بيترجمها. بنرمي exception بنفسنا فقط لما الخطأ يكون من عندنا (مفيش صف users، مفيش

session) — والأفضل تعريف exception خاص بيه بدل استخدام AuthException.

تكرار توست جوجل

الـ initState بيشتغل عند الظهور، والـ BlocListener بيشتغل عند التغيّر فقط. دخول من deep link: الحالة اتغيرت قبل ما الشاشة تظهر → initState بس. تغيّر تاني والشاشة ظاهرة → listener بس. الحل الأنضف: listener واحد على مستوى الجذر داخل MaterialApp.router.builder.

RLS مش بتحمي القيم

الـ policy بتحدد أي صفوف تقدر تلمسها، مش القيم اللي بتبعتها. فمستخدم معدّل يقدر يبعت xp=9999 وهو بيعمل insert لصفه. الحل: trigger + defaults + منع الكتابة، والـ XP/Coins تتحدّث فقط من دوال SECURITY DEFINER.

12) مهام مسجلة

- [x] توست جوجل في مكان واحد: `BlocListener` على الجذر داخل `MaterialApp.router.builder` + `ToastificationWrapper`.
- [x] إزالة `validator` من AppTextField (بقى `TextField`) وإزالة `Form` من signin/credential/name.
- [ ] مراجعة كل القيم الثابتة وتحويلها لـ .sp/.h/.w (مؤجل).

- [ ] مراجعة كل RLS policies + تجربة E2E قبل الـ deploy.
- [ ] تخصيص exceptions للـ domain بدل استعارة `AuthException`.
- [ ] توحيد CharacterRepositoryImpl على FailureMapper.
- [ ] SQL migrations + تأمين أعمدة اللعب في users.
- [ ] استبدال test/widget_test.dart باختبارات حقيقية.
- [ ] حسم شكل الـ Design Tokens (radius / خطوط).